Problem A:

Consider the following two transactions:

***T*34: read(*A*);**

read(*B*);

if *A* = 0 then *B* := *B* + 1;

write(*B*).

***T*35: read(*B*);**

read(*A*);

if *B* = 0 then *A* := *A* + 1;

write(*A*).

Add lock and unlock instructions to transactions *T*34 and *T*35 so that they observe

the two-phase locking protocol. Can the execution of these transactions

result in a deadlock?

Problem B:

Time-stamped based concurrency control is an important concept in database management systems to ensure that concurrent transactions do not violate consistency constraints. The basic idea is to assign a unique timestamp to each transaction and use it to determine the order of transaction execution, ensuring that conflicting operations are executed in a serializable manner.

Let's break down a few common problems and solutions in time-stamped based concurrency control:

Problem 1: Violation of Serialization Order

- **Problem**: Two transactions access the same data items in conflicting ways, and their order is critical to ensure consistency. If transactions execute in an incorrect order, it could lead to incorrect results.
- **Solution**:
    - **Time Stamp Ordering (TSO)**: This is the core concept behind time-stamped based concurrency control. In TSO, each transaction is assigned a unique timestamp, often by the system's clock, when the transaction starts.

- The transactions are ordered based on these timestamps. If transaction T1 starts before T2, then T1 should complete all operations that access the same data item before T2 starts its conflicting operations.
- **Basic Rules**:
    1. If a transaction T1 reads a data item X, the read is valid if no other transaction has written to X after the timestamp of T1.
    2. If a transaction T1 writes to a data item X, the write is valid if no other transaction has read or written to X after the timestamp of T1.
    3. If a transaction violates these rules, it must be aborted and restarted.

Example:

- Transaction T1 starts with timestamp 1, and transaction T2 starts with timestamp 2.
- T1 reads data item X before T2 writes to it. In this case, the read of X by T1 is valid, but if T2 were to write to X after T1's read, the system would abort and restart the conflicting transaction to maintain serializability.

Problem 2: Deadlock in Time-Stamped Systems

- **Problem**: A deadlock can occur when transactions are waiting on each other to release resources in a cyclic fashion. Time-stamped based systems do not inherently handle deadlocks, which may arise due to conflicting operations.
- **Solution**:
    - **Prevention Using Timestamp Ordering**:
        - In time-stamped systems, deadlocks are typically avoided by ensuring that transactions are either executed or aborted based on their timestamps.
        - A transaction with a later timestamp that requests a resource held by a transaction with an earlier timestamp can be aborted immediately to avoid a deadlock.
        - A more common strategy is to use the **wait-die** or **wound-wait** schemes.
            - **Wait-Die**: A younger transaction (with a later timestamp) must wait for an older transaction (with an earlier timestamp) to release the lock. If an older transaction requests a lock held by a younger transaction, the older transaction is aborted.
            - **Wound-Wait**: If a younger transaction requests a lock held by an older transaction, the younger transaction is aborted (wounded). If an older

transaction requests a lock held by a younger transaction, the older transaction waits.

Example:

- Transaction T1 has a timestamp of 1, and T2 has a timestamp of 2.
- T2 requests a lock on a resource held by T1. According to the **Wait-Die** rule, since T2 is younger, it will wait for T1 to finish. If T1 requests a lock held by T2, T1 would be aborted.

Problem 3: Transaction Abort or Restart Overhead

- **Problem**: In time-stamped concurrency control, when transactions violate the timestamp order (due to conflicting operations), they are aborted and restarted. Repeated abortions and restarts can lead to performance overhead.
- **Solution**:
  - **Optimization via Transaction Priority**:
    - If a transaction is continuously being aborted due to conflicts, the system may assign it a lower priority or delay its execution until more favorable conditions arise.
    - **Back-off Algorithms**: These algorithms dynamically adjust the frequency of transactions that are allowed to attempt resource acquisition, reducing the likelihood of repeated conflicts. This can help avoid excessive aborts and retries.

Example:

- Suppose transaction T1 and T2 are repeatedly conflicting, causing frequent restarts. The system might implement a back-off strategy where one of the transactions is delayed or forced to wait for a random period before retrying.

Problem 4: Starvation of Transactions

- **Problem**: In a system that uses time-stamped ordering, a transaction may starve if its timestamp is continuously overtaken by new transactions that keep aborting it.
- **Solution**:
  - **Fairness Strategies**:
    - One solution is to implement a fairness policy where transactions that have been aborted or delayed too many times are given a priority to execute.

- **Transaction Aging**: In this strategy, the timestamp of a transaction increases over time, which helps older transactions (that may have been aborted multiple times) to eventually get executed.

Example:

- Suppose transaction T1 has been aborted several times, and now a new transaction T3 has a higher timestamp, potentially causing T1 to be indefinitely postponed. By aging T1, its effective timestamp is adjusted upward to ensure it gets a fair chance to execute.

Problem 5: Inconsistent Data Reads (Cascading Rollbacks)

- **Problem**: A transaction reads a data item that has been written by another transaction, which later aborts. This can cause cascading rollbacks where many transactions need to be aborted due to dependencies on the aborted transaction.
- **Solution**:
  - **Cascadeless Execution**: One solution to avoid cascading rollbacks is to require transactions to only **read committed data** (i.e., data that has already been written by transactions that have been committed). This ensures that no transaction reads uncommitted data, preventing cascading rollbacks.
  - **Strict Timestamp Ordering**: This approach also limits the possibility of cascading rollbacks by requiring transactions to commit their writes before other transactions can read those values.

Example:

- Transaction T1 writes to data item X, and T2 reads X before T1 commits. If T1 is aborted, T2 may need to be aborted as well, causing a cascading rollback. By ensuring that T2 only reads committed data, this issue is prevented.

Conclusion

Time-stamped based concurrency control is an elegant method to manage concurrent transactions while maintaining serializability, but it comes with its own set of challenges, such as deadlocks, starvation, and overhead due to aborts and restarts. These problems can often be mitigated through various strategies like wait-die, wound-wait, back-off algorithms, transaction aging, and cascadeless execution. Each strategy has its trade-offs, so choosing the right one depends on the system's requirements and workload characteristics.

Problem C:

Scenario-Based Problem: Thomas' Write Rule

Problem Scenario:

Consider a database system managing an inventory of products. Two transactions T1 and T2 are concurrently executing, and both are attempting to modify the quantity of the same product, say "Product X".

- **Transaction T1**:
    - Reads the quantity of Product X (current value is 50).
    - Performs some calculation and decides to increase the quantity by 10.
    - Writes the updated value (60).
- **Transaction T2**:
    - Reads the quantity of Product X (also reads 50).
    - It also performs some calculation and decides to decrease the quantity by 5.
    - Writes the updated value (45).

Now, let's assume that **Transaction T2** is executed after **Transaction T1** (but both are still active at the same time). Without careful concurrency control, a write-write conflict might arise.

Question: How does Thomas' Write Rule resolve this conflict, and how does it ensure correctness and serializability?

Solution Using Thomas' Write Rule:

What is Thomas' Write Rule?

Thomas' Write Rule is a concurrency control rule in **Time-Stamp Ordering (TSO)**, which allows a transaction's write operation to be ignored if it doesn't follow the correct timestamp order. Specifically, Thomas' Write Rule says that:

- **If a transaction T2 writes a data item X, but T2's timestamp is earlier than T1's timestamp (where T1 has already written to X), then T2's write is ignored.**

In other words, **a write by a younger transaction (one with a later timestamp) that conflicts with an older transaction (one with an earlier timestamp) is discarded**, because the older transaction's write is considered the valid one.

Step-by-Step Solution:

1. **Transaction T1** starts with a timestamp of **1** and reads the current quantity of Product X, which is 50. Then, T1 increases the quantity by 10, making the new quantity 60. T1 writes 60 to the database.

2. **Transaction T2** starts with a timestamp of **2** and also reads the current quantity of Product X, which is still 50 (the value seen by T2 is based on the state of the database when it starts).

3. T2 calculates and decides to decrease the quantity by 5, resulting in a value of 45. However, **since T2's timestamp (2) is later than T1's timestamp (1)**, the write by T2 (which would overwrite the write of T1) is **discarded** according to Thomas' Write Rule.

    - **Why is T2's write discarded?**

        - Even though both T1 and T2 read the same value (50) at the time they started, T1 wrote the value 60, and now that T2 comes later (with timestamp 2), it cannot overwrite the results of T1's transaction because T1's update must be considered as the valid one.

4. As a result, **only the write from T1** is committed, and the write from T2 is **ignored** because it does not respect the timestamp ordering required by the time-stamp based concurrency control system.

Final Outcome:

- The final value of Product X is **60** (as written by T1).
- **Transaction T2** is effectively "rolled back" in terms of its write because it attempted to write an invalid value, violating the time-stamp ordering rules.

Problem D:

Time-Stamp Based Ordering (TSO)

Consider a scenario where we have two transactions, T1 and T2, that are accessing the same data item, **Account Balance** (denoted by **A**), in a bank database.

- **Transaction T1** starts at time timestamp1=1.
    - It reads the balance of Account A (initially, A=100).
    - T1 intends to deposit $50 into the account, so it writes the new balance, A=150.
- **Transaction T2** starts at time timestamp2=2.
    - It reads the balance of Account A (which, according to the initial state, is A=100, before T1 has written its update).
    - T2 intends to withdraw $30 from the account, and thus writes the new balance, A=70.

The key question is: **How do we ensure serializability using Time-Stamp Based Ordering (TSO) and what is the final result?**

Solution Using Time-Stamp Based Ordering (TSO)

In Time-Stamp Ordering (TSO), each transaction is assigned a unique timestamp when it starts. The key idea is that each operation (read or write) is compared with the timestamps of other transactions to decide whether the operation should be allowed to proceed or if it should be aborted.

The basic rules for TSO are:

1. **Read Rule**: A transaction T1 is allowed to read data item X if no other transaction T2 has written to X after T1 started. In other words, T1's timestamp should be less than the timestamp of any transaction that has written to X.
2. **Write Rule**: A transaction T1 is allowed to write to data item X if no other transaction T2 has read or written to X after T1 started. Additionally, if T1 writes X, no transaction with a smaller timestamp can write X after T1's write.

Step-by-Step Execution of the Transactions:

- **Step 1: Transaction T1 starts with timestamp 1**.
    - T1 reads the balance of Account A (initially, A=100).
    - T1 proceeds to deposit $50, so it writes A=150 to the database.
- **Step 2: Transaction T2 starts with timestamp 2**.
    - T2 reads the balance of Account A. According to the rules of TSO, it sees the value of A as 100, because the write from T1 has not yet been committed.
    - T2 intends to withdraw $30, so it writes A=70.

At this point, we have a conflict because both T1 and T2 are trying to write to the same data item, **A**, but their operations are in a different order. To resolve this, we apply the **Time-Stamp Ordering Rules**.

Conflict Detection and Resolution:

1. **Conflict between Writes**:
    - **Transaction T1** starts before **Transaction T2** (timestamp 1 for T1 and timestamp 2 for T2).
    - T1 writes to **A**, but T2 also writes to **A** later.
2. **According to TSO Write Rule**:

- Since **T1's timestamp (1)** is earlier than **T2's timestamp (2)**, T1's write takes precedence.
- Therefore, **T2's write is discarded**, and we will **abort** T2 because it violates the time-stamp ordering.

Final Outcome:

- **Transaction T1** writes the updated balance A=150 to the database, and this write is valid since it has the earlier timestamp.
- **Transaction T2** is aborted because it attempts to write a conflicting value (70) after T1's write, and its timestamp is later. The system ensures serializability by discarding the invalid write of T2.

Thus, the final balance of **Account A** after both transactions is **150** (written by T1).

Why Does This Work?

- **Serializability**: The Time-Stamp Ordering protocol ensures that all operations are executed in a serializable manner. By assigning a unique timestamp to each transaction, the system can determine a valid order of operations and prevent conflicting writes.
- **Conflict Resolution**: In this case, T2 is aborted because it violated the time-stamp order by attempting to overwrite the write made by the earlier transaction T1.
- **Correctness**: The final result is consistent with a serial execution of the transactions, where T1 is executed before T2, ensuring that the database ends up in a consistent state.

Key Takeaways:

1. **Time-Stamps Control Execution**: Transactions are executed in timestamp order, which helps resolve conflicts and ensures that the system behaves as if the transactions were executed in a serial order.
2. **Write Conflicts Are Resolved by Aborting**: If two transactions try to write to the same data item, the write from the later transaction is discarded to maintain serializability.
3. **TSO Guarantees Serializability**: By strictly following the timestamp ordering, TSO ensures that the final database state is consistent with a serial execution of the transactions.

This solution illustrates how Time-Stamp Based Ordering (TSO) works to maintain consistency and resolve conflicts in a multi-transaction environment.

Problem E:

To illustrate the Time-Stamp Ordering (TSO) Protocol, we'll prepare a set of schedules involving multiple transactions. The idea is to show how the protocol uses timestamps to determine the correct serial order of operations and handle conflicts between transactions.

We'll use two transactions in our example, T1 and T2, that both access a data item X.

Example Setup:

- Data item: X.
- Initially, X=100.
- Transaction T1 has timestamp TS1=1 (it starts first).
- Transaction T2 has timestamp TS2=2 (it starts later).

The operations will be as follows:

- T1 reads X, then writes X=150.
- T2 reads X, then writes X=50.

We'll demonstrate different scenarios to show how Time-Stamp Ordering (TSO) resolves conflicts.

Schedule 1: Serial Execution (No Conflict)

Operations:

- T1 reads XXX.
- T1 writes X=150.
- T2 reads X=150 (the updated value from T1).
- T2 writes X=50.

Schedule:

Time	Transaction	Operation	Data Item	Value
1	T1	Read(X)	X	100
2	T1	Write(X)	X	150
3	T2	Read(X)	X	150

| 4 | T2 | Write(X) | X | 50 |

Analysis:

- This is a serializable schedule, where T1 runs to completion before T2 starts.
- T2 sees the updated value of X=150 written by T1.
- There is no conflict, so the final value of X after both transactions is X=50, the last write by TT2 .

Schedule 2: T1 and T2 Conflicting Writes (TSO Resolves)

Operations:

- T1 reads X.
- T1 writes X=150.
- T2 reads X.
- T2 writes X=50.

Schedule:

| Time | Transaction | Operation | Data Item | Value |

|---|---|---|---|---|

| 1 | T1 | Read(X) | X | 100 |

| 2 | T1 | Write(X) | X | 150 |

| 3 | T2 | Read(X) | X | 100 |

| 4 | T2 | Write(X) | X | 50 |

Analysis (Time-Stamp Based Ordering Applied):

- T1's timestamp is earlier than T2's, so T1's write must be considered valid.
- T1 writes X=150to the database.
- Conflict: T2 reads X=100 (before T1's write), but it tries to write X=50, which conflicts with T1's earlier write of X=150.
    - According to TSO, since T2's timestamp (2) is later than T1's timestamp (1), T2's write is ignored.
    - T2 will be aborted, and it will not be allowed to overwrite the value of X=150 written by T1.

Result:

- The final value of X is 150, written by T1, and T2 is aborted.

Schedule 3: T1 Reads, T2 Writes (Without Conflict)

Operations:

- T1 reads X.
- T1 writes X=150.
- T2 reads X=150 (the updated value from T1).
- T2 writes X=50.

Schedule:

Time	Transaction	Operation	Data Item	Value
1	T1	Read(X)	X	100
2	T1	Write(X)	X	150
3	T2	Read(X)	X	150
4	T2	Write(X)	X	50

Analysis:

- This schedule is serializable and follows the correct order based on timestamps.
- T1 performs the write first, and T2 follows, reading the value written by T1.
- The final value of X is 50, the last write from T2.

Schedule 4: T2 Conflicts with T1's Write (Aborted by TSO)

Operations:

- T1 reads X.
- T1 writes X=150.
- T2 reads X.
- T2 writes X=50.

- T1 reads X again and writes X=200.

Schedule:

Time	Transaction	Operation	Data Item	Value
1	T1	Read(X)	X	100
2	T1	Write(X)	X	150
3	T2	Read(X)	X	100
4	T2	Write(X)	X	50
5	T1	Read(X)	X	50
6	T1	Write(X)	X	200

Analysis:

- Conflict Between Reads and Writes: T1 writes X=150, and then T2 writes X=50, which conflicts with T1's earlier write.
- TSO Rule: Since T2's timestamp (2) is later than T1's timestamp (1), T2's write is discarded, and T2 is aborted.
- Final Value of X: The valid write is from T1, so the final value of X is X=200.

Summary of Time-Stamp Ordering Behavior:

1. Serial Execution (Schedule 1) – No conflict: Transactions execute one after the other with no issues.
2. Write Conflicts (Schedule 2) – Conflict resolution: TSO ensures that the transaction with the earlier timestamp (here, T1) wins and the later transaction (here, T2) is aborted.
3. Read/Write Conflict (Schedule 3) – No conflict when reading the most recent value: The read operations do not conflict with writes.
4. Aborted Transaction (Schedule 4) – A transaction that conflicts with an earlier write is aborted, ensuring consistency and serializability.

Key Concepts in Time-Stamp Ordering (TSO):

1. Conflict Resolution: When two transactions conflict (e.g., both write to the same item), the transaction with the earlier timestamp is allowed to proceed, while the later transaction is aborted.

2. Timestamp-Based Order: Every transaction is assigned a timestamp at the beginning. This timestamp helps determine the correct order of operations to ensure serializability.
3. Serializability: TSO ensures that the final outcome of a set of transactions is equivalent to some serial order of those transactions.